

# DBMS – WEEK 5

## PET SHOP MANAGEMENT SYSTEM

### Payment Transaction Management

#### Scenario:

The pet shop management system needs to maintain payment information for customer orders. The system should record payment details, track transaction status, manage successful and failed payments, and generate payment-related reports. The Payment Transaction Module is connected with the existing Customer and Order Management modules.

**Objective:** To manage customer payment transactions and analyze payment activity in the Pet Shop Management System.

#### Tasks to be Done

- Create Payment Table
- Store Payment Details
- Manage Successful and Failed Transactions
- Analyze Payment Methods Used by Customers
- Generate Payment-related Reports

#### 1. Create Payment Table

The Payment table stores payment transaction details for orders placed in the pet shop.

| Attribute          | Description                     | Example    |
|--------------------|---------------------------------|------------|
| Payment ID         | Unique payment identifier       | PAY001     |
| Order ID           | Order reference                 | ORD001     |
| Payment Date       | Date of transaction             | 01-10-2026 |
| Payment Mode       | UPI / Card / Net Banking / Cash | UPI        |
| Payment Status     | Success / Failed / Pending      | Success    |
| Transaction Amount | Amount paid for the order       | 2500       |

```
SQL> CREATE TABLE Payment (  
  2     Payment_ID VARCHAR(7),  
  3     Order_ID VARCHAR(7),  
  4     Payment_Date DATE,  
  5     Payment_Mode VARCHAR(15),  
  6     Payment_Status VARCHAR(10),  
  7     Transaction_Amount NUMBER  
  8 );
```

Table created.

## 2. Add Primary Key

Payment\_ID is made the primary key so that every payment transaction has a unique identifier.

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT PK_Payment PRIMARY KEY(Payment_ID);

Table altered.
```

## 3. Add Foreign Key Reference to Order Table

Order\_ID in Payment references Order\_ID in the existing Orders table. This ensures that a payment can only be recorded for a valid order.

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT FK_Payment_Order
  3  FOREIGN KEY (Order_ID)
  4  REFERENCES Orders(Order_ID);

Table altered.
```

Note: The Orders table must already exist, and Orders(Order\_ID) must be a Primary Key or have a UNIQUE constraint.

## 4. Store Payment Details

The following 10 payment records represent transactions made by customers of the pet shop.

```

SQL> INSERT INTO Payment VALUES ('PAY001','0001',DATE '2026-10-01','UPI','Success',2500);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY002','0002',DATE '2026-10-01','Card','Success',1800);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY003','0003',DATE '2026-10-02','Cash','Success',1200);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY004','0004',DATE '2026-10-02','UPI','Failed',3000);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY005','0005',DATE '2026-10-03','Card','Success',2200);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY006','0006',DATE '2026-10-03','UPI','Success',1500);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY007','0007',DATE '2026-10-04','Cash','Failed',900);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY008','0008',DATE '2026-10-04','UPI','Success',3500);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY009','0009',DATE '2026-10-05','Card','Success',2800);
1 row created.

SQL> INSERT INTO Payment VALUES ('PAY010','0010',DATE '2026-10-05','UPI','Pending',1700);
1 row created.

SQL>
SQL> COMMIT;

Commit complete.

```

```

SQL> select * from payment;

```

| PAYMENT | ORDER_I | PAYMENT_D | PAYMENT_MODE | PAYMENT_ST | TRANSACTION_AMOUNT |
|---------|---------|-----------|--------------|------------|--------------------|
| -----   | -----   | -----     | -----        | -----      | -----              |
| PAY001  | 0001    | 01-OCT-26 | UPI          | Success    | 2500               |
| PAY002  | 0002    | 01-OCT-26 | Card         | Success    | 1800               |
| PAY003  | 0003    | 02-OCT-26 | Cash         | Success    | 1200               |
| PAY004  | 0004    | 02-OCT-26 | UPI          | Failed     | 3000               |
| PAY005  | 0005    | 03-OCT-26 | Card         | Success    | 2200               |
| PAY006  | 0006    | 03-OCT-26 | UPI          | Success    | 1500               |
| PAY007  | 0007    | 04-OCT-26 | Cash         | Failed     | 900                |
| PAY008  | 0008    | 04-OCT-26 | UPI          | Success    | 3500               |
| PAY009  | 0009    | 05-OCT-26 | Card         | Success    | 2800               |
| PAY010  | 0010    | 05-OCT-26 | UPI          | Pending    | 1700               |

```

10 rows selected.

```

## 5. Manage Successful and Failed Transactions

### a) Display all successful payments

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Success';
```

| PAYMENT | ORDER_I | PAYMENT_D | PAYMENT_MODE | PAYMENT_ST | TRANSACTION_AMOUNT |
|---------|---------|-----------|--------------|------------|--------------------|
| PAY001  | 0001    | 01-OCT-26 | UPI          | Success    | 2500               |
| PAY002  | 0002    | 01-OCT-26 | Card         | Success    | 1800               |
| PAY003  | 0003    | 02-OCT-26 | Cash         | Success    | 1200               |
| PAY005  | 0005    | 03-OCT-26 | Card         | Success    | 2200               |
| PAY006  | 0006    | 03-OCT-26 | UPI          | Success    | 1500               |
| PAY008  | 0008    | 04-OCT-26 | UPI          | Success    | 3500               |
| PAY009  | 0009    | 05-OCT-26 | Card         | Success    | 2800               |

7 rows selected.

### b) Find failed transactions

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Failed';
```

| PAYMENT | ORDER_I | PAYMENT_D | PAYMENT_MODE | PAYMENT_ST | TRANSACTION_AMOUNT |
|---------|---------|-----------|--------------|------------|--------------------|
| PAY004  | 0004    | 02-OCT-26 | UPI          | Failed     | 3000               |
| PAY007  | 0007    | 04-OCT-26 | Cash         | Failed     | 900                |

### c) Count total successful and failed payments

```
SQL> SELECT Payment_Status,
  2          COUNT(*) AS Total_Transactions
  3  FROM Payment
  4  WHERE Payment_Status IN ('Success', 'Failed')
  5  GROUP BY Payment_Status;
```

| PAYMENT_ST | TOTAL_TRANSACTIONS |
|------------|--------------------|
| Failed     | 2                  |
| Success    | 7                  |

## 6. Analyze Payment Methods Used by Customers

Payment analysis reports help the pet shop understand which payment methods customers prefer and how transactions are distributed across different payment modes.

### Report 1: Payment Mode Analysis

- Number of transactions using UPI
- Number of card payments
- Most preferred payment method

#### Number of UPI Transactions

```
SQL> SELECT COUNT(*) AS UPI_Transactions
2  FROM Payment
3  WHERE Payment_Mode = 'UPI';
```

```
UPI_TRANSACTIONS
-----
5
```

#### Number of Card Payments

```
SQL> SELECT COUNT(*) AS Card_Payments
2  FROM Payment
3  WHERE Payment_Mode = 'Card';
```

```
CARD_PAYMENTS
-----
3
```

#### Most Preferred Payment Method

```
SQL> SELECT Payment_Mode,
2  COUNT(*) AS Number_of_Transactions
3  FROM Payment
4  GROUP BY Payment_Mode
5  ORDER BY COUNT(*) DESC;
```

```
PAYMENT_MODE    NUMBER_OF_TRANSACTIONS
-----
UPI              5
Card             3
Cash             2
```

## Report 2: Revenue Analysis

- Total revenue generated
- Revenue by payment method
- Average transaction amount

### Total Revenue Generated

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue
2 FROM Payment
3 WHERE Payment_Status = 'Success';

TOTAL_REVENUE
-----
          15500
```

### Revenue by Payment Method

```
SQL> SELECT Payment_Mode,
2          SUM(Transaction_Amount) AS Revenue
3 FROM Payment
4 WHERE Payment_Status = 'Success'
5 GROUP BY Payment_Mode;

PAYMENT_MODE      REVENUE
-----
Card              6800
Cash              1200
UPI               7500
```

### Average Transaction Amount

```
SQL> SELECT AVG(Transaction_Amount) AS Average_Transaction_Amount
2 FROM Payment
3 WHERE Payment_Status = 'Success';

AVERAGE_TRANSACTION_AMOUNT
-----
          2214.28571
```

## Report 3: Customer Payment History

This report displays payment information together with the customer and order details. The exact Customer and Order column names should match the existing tables in the Pet Shop Management System.

- Customer name
- Order ID

- Payment mode
- Transaction amount
- Payment status

```
SQL> SELECT c.First_Name,
2         c.Last_Name,
3         o.Order_ID,
4         p.Payment_Mode,
5         p.Transaction_Amount,
6         p.Payment_Status
7 FROM Customer c
8 JOIN Orders o
9 ON c.Customer_ID =
10      CASE o.Customer_ID
11         WHEN 'C001' THEN 'S101'
12         WHEN 'C002' THEN 'S102'
13         WHEN 'C003' THEN 'S103'
14         WHEN 'C004' THEN 'S104'
15         WHEN 'C005' THEN 'S105'
16      END
17 JOIN Payment p
18 ON o.Order_ID = p.Order_ID;
```

| FIRST_NAME | LAST_NAME    | ORDER_ID |
|------------|--------------|----------|
| AKSHAYA    | NIVASINI     | 0001     |
| UPI        | 2500 Success |          |
| PRITHISH   | RAJ          | 0002     |
| CARD       | 1800 Success |          |
| KSHAUNISH  | RAJESH       | 0003     |
| CASH       | 1200 Success |          |

  

| FIRST_NAME | LAST_NAME    | ORDER_ID |
|------------|--------------|----------|
| KANISKA    | RAM          | 0004     |
| UPI        | 3000 Failed  |          |
| RUKMANI    | KUPPUSAMY    | 0005     |
| CARD       | 2200 Success |          |
| AKSHAYA    | NIVASINI     | 0006     |
| UPI        | 1500 Success |          |

  

| FIRST_NAME | LAST_NAME  | ORDER_ID |
|------------|------------|----------|
| PRITHISH   | RAJ        | 0007     |
| CASH       | 900 Failed |          |

|                   |                           |            |
|-------------------|---------------------------|------------|
| kshaunish<br>UPI  | Rajesh<br>3500 Success    | 0008       |
| kaniska<br>Card   | Ram<br>2800 Success       | 0009       |
| -----             |                           |            |
| FIRST_NAME        | LAST_NAME                 | ORDER_ID   |
| -----             |                           |            |
| PAYMENT_MODE      | TRANSACTION_AMOUNT        | PAYMENT_ST |
| -----             |                           |            |
| Rukmani<br>UPI    | Kuppusamy<br>1700 Pending | 0010       |
| 10 rows selected. |                           |            |

Note: If your existing order table is named Order instead of Orders, use the exact table name already used in your project.

### PAYMENT ANALYSIS:

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue
2 FROM Payment
3 WHERE Payment_Status = 'Success';

TOTAL_REVENUE
-----
          15500

SQL> SELECT Payment_Mode,
2         SUM(Transaction_Amount) AS Revenue
3 FROM Payment
4 WHERE Payment_Status = 'Success'
5 GROUP BY Payment_Mode;

PAYMENT_MODE      REVENUE
-----
Card              6800
Cash              1200
UPI               7500

SQL> SELECT AVG(Transaction_Amount) AS Average_Transaction_Amount
2 FROM Payment
3 WHERE Payment_Status = 'Success';

AVERAGE_TRANSACTION_AMOUNT
-----
          2214.28571
```

### 6. Summary

The Payment Transaction Management module records payment details for pet shop orders, identifies successful and failed transactions, analyzes payment methods, and generates revenue and customer payment reports. These operations help the pet shop maintain accurate transaction records and understand its payment activity.

### Expected Outcome

- Payment transactions are stored systematically.
- Successful, failed, and pending payments can be identified.

- UPI, card, and cash payment usage can be analyzed.
- Total and average successful transaction amounts can be calculated.
- Customer payment history can be retrieved using joins with existing project tables.